

32 - OpenET 2: Data Collection and Long-Term Benchmarking Tools for Remote Eye-Tracking Devices

Client: Dr Peter Wagner

Allowed groups: 2 groups

Project background & goals

This project builds on a previous capstone project titled OpenET: A Unified Python Framework for Benchmarking Remote Eye-Tracking Devices.

Eye-tracking devices are increasingly used in research, digital health, optometry, human-computer interaction, accessibility, education, virtual reality, and clinical applications. However, different remote eye-tracking devices often use different data formats, calibration procedures, sampling rates, quality indicators, and export structures. This makes it difficult to compare devices in a systematic, repeatable, and long-term way.

The previous OpenET project established the foundation for a reusable Python framework for importing, processing, and benchmarking eye-tracking data. The next stage is to complete and extend this framework by developing practical tools for standardised data collection, session management, device metadata handling, automated quality checks, and long-term comparison of eye-tracking device performance.

The goal of this project is to make OpenET more research-ready by supporting structured and repeatable benchmarking of remote eye-tracking devices across different sessions, devices, test conditions, and users. The final system should help researchers collect, organise, validate, visualise, and compare eye-tracking data over time.

This project provides students with an opportunity to contribute to a real-world research and digital health challenge while developing a reusable software tool with potential for future research and clinical applications.

Project scope

The project will focus on extending and completing the existing OpenET framework. The main emphasis is on building software tools that support standardised data collection and long-term benchmarking of remote eye-tracking devices.

The project scope includes:

- Reviewing the existing OpenET framework and identifying areas that require completion or improvement.

- Designing a structured data collection workflow for eye-tracking device benchmarking.

- Developing session and device management tools, including device IDs, session IDs, participant/test IDs, calibration metadata, and recording information.

- Supporting import and management of eye-tracking data from multiple devices or file formats.

- Developing tools for standardised benchmark recordings, including start/stop events, test conditions, and metadata logging.

- Implementing automated data quality checks for eye-tracking recordings.

- Calculating relevant eye-tracking performance metrics such as sampling rate, data completeness, missing samples, precision, stability, and calibration-related indicators.

- Creating visualisation tools for inspecting recordings and comparing device performance.

- Supporting repeated measurements over time to enable long-term device comparison.

Producing clear user and developer documentation.

The project should be achievable for a group of approximately six students over ten weeks. The expected outcome is a functional, maintainable, and well-documented research software system rather than a commercial product.

Project requirements

Key required features include:

1. Review and extension of the existing OpenET framework

Students should review the previous OpenET codebase and extend it using a clean and maintainable software architecture. The system should remain modular so that future devices, data formats, test protocols, and analysis modules can be added.

2. Structured session management

The system should support structured benchmarking sessions, including session IDs, device IDs, test IDs, timestamps, calibration information, trial numbers, test conditions, and relevant metadata. The software should create or manage consistent folder structures for recordings.

3. Data collection workflow

The software should support a repeatable data collection workflow for benchmarking remote eye-tracking devices. This may include a guided sequence of calibration, validation, recording, test stimulus presentation, start/stop logging, and session completion.

4. Device and metadata management

The system should allow researchers to record information about the eye-tracking device, software version, sampling rate, calibration settings, participant/test setup, monitor/display setup, and environmental conditions where relevant.

5. Multi-format data import

The system should support importing eye-tracking data from different devices or export formats, such as CSV, TSV, JSON, or other structured text formats. The design should allow new import modules to be added later.

6. Automated data quality checks

The system should automatically detect common issues in eye-tracking recordings, including missing samples, duplicate timestamps, irregular sampling frequency, missing gaze coordinates, invalid samples, incomplete recordings, and inconsistent metadata.

7. Benchmarking and comparison metrics

The system should calculate and summarise relevant comparison metrics, such as recording duration, effective sampling rate, data loss, gaze precision, fixation stability, calibration quality indicators, and consistency across repeated recordings.

8. Visualisation tools

The software should provide visual outputs for reviewing individual recordings and comparing devices. Useful visualisations may include gaze traces, fixation stability plots, missing-data timelines, sampling frequency plots, calibration-point summaries, and device comparison charts.

9. Long-term comparison support

The system should support repeated benchmarking sessions across multiple days, weeks, or testing periods. This should allow researchers to track whether device performance changes over time.

10. Documentation and usability

The final system should include installation instructions, user documentation, developer documentation, example datasets, and example workflows.

Required knowledge and skills

Students should have an interest in software development, data processing, and applied research technology.

Useful skills include:

- Python programming
- Modular software design
- Working with CSV, TSV, JSON, and structured data
- Data visualisation
- Time-series data processing
- Git/GitHub version control
- Testing and debugging
- Basic user interface or dashboard development
- Interest in eye tracking, digital health, human-computer interaction, vision science, or biomedical applications

Helpful but not essential:

- Experience with eye-tracking data
- Experience with experimental data collection
- Experience with sensor data
- Experience with web dashboards or graphical user interfaces
- Basic understanding of statistics or signal processing

Students do not need prior knowledge of optometry or vision science, but they should be willing to work with real research data and practical experimental requirements.

Expected outcomes and deliverables

Expected deliverables include:

- Extended OpenET source code
- Structured data collection workflow for benchmarking remote eye-tracking devices
- Session, device, and metadata management tools
- Data import modules for eye-tracking files

Automated data quality and validation checks
Benchmarking and comparison metrics
Visualisation tools for individual recordings and device comparisons
Support for repeated/long-term benchmarking
Example datasets and example workflows
Installation instructions
User guide
Developer documentation
Final technical report
Demonstration of the system using example or real eye-tracking datasets

Disciplines related to the project

Software Development;Computer Science and Algorithms;Big data Analytics and Visualization;Human Computer Interaction (HCI);Bioinformatics/Biomedical

Resources provided

Students will be provided with research context, supervision, and example data related to the OpenET project.

Resources may include:

Existing OpenET prototype/source code from the previous capstone project
Example eye-tracking datasets
Example device export files
Example calibration and validation data
Expected folder structures and metadata formats
Guidance on relevant eye-tracking quality metrics
Regular feedback from the project supervisor
Access to selected eye-tracking devices or recorded datasets, depending on feasibility and project logistics

The project should be designed so that core development can proceed using provided example datasets, even if direct access to all devices is limited.